Gamepad Control on PayloadSDK

Gamepad Control

- Control the gimbal and camera through PayloadSDK with a Logitech F310 gamepad.
- Channels (which button/axis does what) can be changed while the program is running by typing commands in the terminal.

1. Features

Action	Type	Default input	Description
`GIMBAL_YAW`	analog	`LX`	Rotate gimbal yaw
`GIMBAL_PITCH`	analog	`LY`	Rotate gimbal pitch
`GIMBAL_ROLL`	analog	-	Rotate gimbal roll (not assigned by default)
`ZOOM`	analog	`RY`	Push up = zoom in, push down = zoom out, release = stop
`GIMBAL_HOME`	one-shot	`A`	Return gimbal to home position
`IMAGE_CAPTURE`	one-shot	`B`	Take a photo
`VIDEO_RECORD`	one-shot	`X`	Start / stop video recording
`CHANGE_VIEW`	one-shot	`Y`	Cycle view source
`CHANGE_RECORD`	one-shot	`LB`	Cycle record source
`GIMBAL_MODE`	one-shot	`RB`	Cycle gimbal mode

Action	Type	Default input	Description
`TRACKING`	one-shot	`LOGITECH`	Start / stop object tracking

- **Analog**: needs a continuous value. Can only be set on a stick (`LX LY RX RY`).
- **One-shot**: runs once per press. Can be set on any button or axis.
  An axis used as a button counts as one press when pushed past half its travel (joystick, D-pad) or squeezed past half (`LT`, `RT`).
- **Button names**: `A B X Y LB RB BACK START LOGITECH L3 R3`
- **Axis names**: `LX LY LT RX RY RT DPAD_X DPAD_Y`

2. Build

```
mkdir build && cd build
cmake -Dpayload_type ..     (e.g. cmake -DVIO=1 ..)
make -j$(nproc)
```

3. Run

```
./tests/gamepad_control/gamepad_ctrl
```

4. Commands

Commands are not case sensitive (`set rx gimbal_yaw` = `SET RX GIMBAL_YAW`).

Command	Description
`help` or `h`	Show all commands
`show`	Show current mapping (inputs set to `NONE` are hidden)
`list`	List all button, axis and action names
`set <BUTTON|AXIS> <ACTION>`	Set the action of a button/axis
`clear <BUTTON|AXIS>`	Clear the action of a button/axis (same as `set ... NONE`)

Command	Description
`reset`	Restore the default mapping
`log on|off`	Print raw gamepad events (useful to find button/axis names)
`quit` or `q`	Exit program

5.Command usage

`help` / `h`: show all commands

```
> help
Commands:
  show                        Show current mapping
  list                        List button, axis and action names
  set <BUTTON|AXIS> <ACTION>  Set the action of a button/axis, e.g. set RX GIMBAL_YAW | set LB
IMAGE_CAPTURE
  clear <BUTTON|AXIS>         Clear the action of a button/axis (same as set ... NONE)
  reset                       Restore default mapping
  log on|off                  Print gamepad events on/off (to see button/axis names)
  quit                        Exit program
```

`show`: current mapping

```
> show
---- Button ----
  A        -> GIMBAL_HOME
  B        -> IMAGE_CAPTURE
  X        -> VIDEO_RECORD
  Y        -> CHANGE_VIEW
  LB       -> CHANGE_RECORD
  RB       -> GIMBAL_MODE
  LOGITECH -> TRACKING
---- Axis ----
  LX       -> GIMBAL_YAW
```

```
  LY        -> GIMBAL_PITCH
  RY        -> ZOOM
```

Inputs not listed are `NONE` (do nothing).

`list`: all names that can be used with `set` / `clear`

```
> list
Buttons: A  B  X  Y  LB  RB  BACK  START  LOGITECH  L3  R3
Axes   : LX  LY  LT  RX  RY  RT  DPAD_X  DPAD_Y
Analog actions (axis only):
  GIMBAL_PITCH  GIMBAL_YAW  GIMBAL_ROLL  ZOOM
One-shot actions (any button or axis):
  GIMBAL_HOME  GIMBAL_MODE  IMAGE_CAPTURE  VIDEO_RECORD  CHANGE_VIEW  CHANGE_RECORD  TRACKING
```

`set <BUTTON|AXIS> <ACTION>`: change a channel

Move `GIMBAL_YAW` from the left joystick (`LX`) to the right joystick (`RX`):

```
> set RX GIMBAL_YAW
  RX -> GIMBAL_YAW
```

An action can only be on one input at a time, so `LX` is cleared to `NONE` automatically.

Take photos with the right trigger (lowercase also works):

```
> set rt image_capture
  RT -> IMAGE_CAPTURE
```

Errors:

```
> set A ZOOM
  ZOOM is an analog action, it can only be set on an axis
> set QQ ZOOM
  'QQ' is not a button or axis (type 'list')
> set RX FLY
  'FLY' is not an action (type 'list')
> set RX
```

```
  Usage: set <BUTTON|AXIS> <ACTION>
```

When a command fails, the mapping is not changed.

`clear <BUTTON|AXIS>`: remove an action

```
> clear B
  B -> NONE
```

Button `B` now does nothing. Same as `set B NONE`.

`reset`: restore the default mapping

```
> reset
  Restored default mapping
---- Button ----
  A        -> GIMBAL_HOME
  ...
```

`load`: reload the mapping file

After editing `gamepad_mapping.txt` by hand while the program is running:

```
> load
Loaded mapping from gamepad_mapping.txt
---- Button ----
  A        -> GIMBAL_HOME
  ...
```

`log on` / `log off`: print raw gamepad events

Useful to find the name of a button or axis, or to check raw stick values.

```
> log on
  Event log: ON
Button LB pressed
CHANGE_RECORD -> LB
```

```
Set record source to 1
Button LB released
Axis LX = 8452
Axis LX = 15320
Axis LX = 0
> log off
  Event log: OFF
```

`CHANGE_RECORD -> LB` is printed by `executeAction()` for every one-shot action, even when the log is off.

`quit` / `q`: exit

```
> quit
Exit
```

`Ctrl+C` does the same. Gimbal speed and zoom are stopped before exiting.

Unknown command

```
> foo
  Unknown command: FOO
Commands:
  ...
```

Mapping file

The current mapping is stored in `gamepad_mapping.txt`, in the directory the program is started from.

- **First start**: the file does not exist, so it is created from the default mapping (`DEFAULT_BUTTON_MAP` / `DEFAULT_AXIS_MAP` in `gamepad_controller.h`).
- **Next starts**: the file is loaded on top of the default mapping.
- **`set`** / **`clear`** / **`reset`**: the file is saved immediately.
- **Edit by hand**: open the file, change it, then restart the program or type `load`.

```
A = GIMBAL_HOME
B = IMAGE_CAPTURE
```

```
X = VIDEO_RECORD
...

LX = GIMBAL_YAW
LY = GIMBAL_PITCH
...
```

Rules:

- One `INPUT = ACTION` per line. Names are not case sensitive. Text after `#` is a comment.
- Inputs missing from the file keep their default action.
- Wrong lines are skipped with a warning, the rest of the file is still loaded:

```
WARNING: gamepad_mapping.txt line 13: TRACKING is used twice, only the last one (LOGITECH) is
kept
WARNING: gamepad_mapping.txt line 14: unknown name, skipped
WARNING: gamepad_mapping.txt line 15: ZOOM cannot be set on R3, skipped
```

- To go back to the defaults: type `reset`, or delete the file and restart.
- To use another file: `my_gamepad->setMappingFile("/path/to/file.txt");` before `start()` in `main.cpp`.

6. Files

File	Description
`gamepad_reader.h/.cpp`	Reads `/dev/input/jsN`, auto-detects and reconnects the gamepad
`gamepad_controller.h/.cpp`	Actions, mapping table, CLI
`main.cpp`	`executeAction()`: turns each action into PayloadSDK calls

7. Adding a new feature

Example 1: one-shot action `ZOOM_TELE`

1. Add the action to `enum class Action` (`gamepad_controller.h`):

```
    TRACKING,
    ZOOM_TELE,

    COUNT
};
```

2. Add its name to `kActionNames` (`gamepad_controller.h`), so it can be typed in the terminal:

```
    { Action::ZOOM_TELE, "ZOOM_TELE" },
```

3. Skip (one-shot actions are not added to `isAnalog()`).

4. Handle it in `executeAction()` (`main.cpp`). Called once per press:

```
    case Action::ZOOM_TELE:
        // call PayloadSDK here
        break;
```

5. (Optional) Set a default button in `DEFAULT_BUTTON_MAP` (`gamepad_controller.h`). Otherwise assign it at runtime with `set <INPUT> ZOOM_TELE`.

```
    /* L3       */ Action::ZOOM_TELE,
```

Example 2: analog action `FOCUS`

1. Add the action to `enum class Action` (`gamepad_controller.h`):

```
    TRACKING,
    FOCUS,

    COUNT
};
```

2. Add its name to `kActionNames` (`gamepad_controller.h`):

```
    { Action::FOCUS, "FOCUS" },
```

3. Add a `case` **to** `GamepadController::isAnalog()` (`gamepad_controller.cpp`):

```
bool GamepadController::isAnalog(Action a) {
    switch (a) {
    case Action::GIMBAL_PITCH:
    case Action::GIMBAL_YAW:
    case Action::GIMBAL_ROLL:
    case Action::ZOOM:
    case Action::FOCUS:          // <- add here
        return true;
    default:
        return false;
    }
}
```

4. Handle it in `executeAction()` (`main.cpp`). Called whenever the stick value changes. Read the value with `getAnalog()` (range `-32767..32767`, `0` = released):

```
    case Action::FOCUS: {
        int f = applyDeadzone(my_gamepad->getAnalog(Action::FOCUS));
        // call PayloadSDK here
        break;
    }
```

5. (Optional) Set a default axis in `DEFAULT_AXIS_MAP` (`gamepad_controller.h`). Otherwise assign it at runtime with `set <AXIS> FOCUS`.

```
    /* RX     */ Action::FOCUS,
```

The order of `DEFAULT_BUTTON_MAP` / `DEFAULT_AXIS_MAP` must match `enum GamepadButton` / `enum GamepadAxis` in `gamepad_reader.h`. The `/* A */` comments are only labels; the compiler uses the position.

8. Adjustable constants

Constant	File	Default	Meaning
`JOYSTICK_DEADZONE`	`main.cpp`	2500	Joystick values below this are treated as 0

Constant	File	Default	Meaning
`GIMBAL_PITCH_SPEED`	`main.cpp`	20	Max pitch speed (deg/s) at full joystick
`GIMBAL_YAW_SPEED`	`main.cpp`	90	Max yaw speed (deg/s) at full joystick
`GIMBAL_ROLL_SPEED`	`main.cpp`	20	Max roll speed (deg/s) at full joystick
`update(20)`	`main.cpp`	20 ms	Loop period: lower = faster response, more commands

Revision #13
Created 2026-10-22 07:44:04 UTC by Trần Văn Hồ
Updated 2026-10-26 02:49:09 UTC by Trần Văn Hồ